

소프트웨어프로젝트2 도전과제 (09차시)

09C19 | 초음파 센서 중위수 필터

1. 동작 방식

25 ms마다 초음파 센서의 거리를 mm 단위로 측정한다.

원본 값(raw)은 최근 N개를 저장하는 원형 버퍼에 넣는다.

버퍼를 복사해 오름차순으로 정렬한 뒤 가운데 값을 median으로 사용한다.

표본 수가 짝수이면 가운데 두 값의 평균을 사용한다.

2. N 설정 방법

코드 상단의 #define MEDIAN_N 3에서 숫자를 3, 10, 30으로 바꾼다.

각 설정마다 보드에 업로드한 뒤 시리얼 플로터를 57600 baud로 연다.

시작 직후에는 저장된 표본 수만큼 계산하며, N개가 모이면 최근 N개만 쓴다.

3. 오류값 처리와 EMA

pulseIn()의 timeout 결과인 0도 원본 표본으로 버퍼에 저장한다.

측정 오류를 직전 유효값으로 대체하는 처리는 사용하지 않는다.

EMA는 raw를 이용해 별도로 계산하며 중위수 필터의 입력으로 쓰지 않는다.

$EMA = 0.5 \times \text{현재 raw} + 0.5 \times \text{이전 EMA}$

4. 시리얼 출력 형식

Min:100,raw:측정값,ema:EMA값,median:중위수값,Max:300

Min, raw, ema, median, Max 순서로 한 줄씩 출력한다.

5. 연결 및 비교 방법

TRIG: 12번, ECHO: 13번, 센서 전원: 5V 및 공통 GND.

LED는 9번 핀을 사용하며 Active LOW 방식이다. 외부 LED에는 저항이 필요하다.

동일한 움직임으로 N=3, 10, 30의 실제 시리얼 플로터 화면을 비교한다.

N이 커질수록 짧게 튀는 값은 줄일 수 있지만 거리 변화 반응이 늦어진다.

코드 원본 | 09_challenge.ino

```
#define PIN_LED 9
#define PIN_TRIG 12
#define PIN_ECHO 13
#define SND_VEL 346.0
#define INTERVAL 25
#define PULSE_DURATION 10
#define _DIST_MIN 100
#define _DIST_MAX 300
#define TIMEOUT ((INTERVAL / 2.0) * 1000.0)
#define SCALE (0.001 * 0.5 * SND_VEL)
#define _EMA_ALPHA 0.5
#define MEDIAN_N 3

unsigned long last_sampling_time = 0;
float dist_ema = 0.0;
float samples[MEDIAN_N];
int sample_index = 0;
int sample_count = 0;

void setup() {
    pinMode(PIN_LED, OUTPUT);
    pinMode(PIN_TRIG, OUTPUT);
    pinMode(PIN_ECHO, INPUT);
    digitalWrite(PIN_TRIG, LOW);
    digitalWrite(PIN_LED, HIGH);
    Serial.begin(57600);
    last_sampling_time = millis();
}

void loop() {
    unsigned long now = millis();
    if (now - last_sampling_time < INTERVAL) return;
    last_sampling_time = now;

    float dist_raw = USS_measure(PIN_TRIG, PIN_ECHO);
    if (sample_count == 0) dist_ema = dist_raw;
    dist_ema = _EMA_ALPHA * dist_raw + (1.0 - _EMA_ALPHA) *
dist_ema;

    samples[sample_index] = dist_raw;
    sample_index = (sample_index + 1) % MEDIAN_N;
```

```
    if (sample_count < MEDIAN_N) sample_count++;
    float dist_median = getMedian(samples, sample_count);

    Serial.print("Min:");
    Serial.print(_DIST_MIN);
    Serial.print(",raw:");
    Serial.print(dist_raw);
    Serial.print(",ema:");
    Serial.print(dist_ema);
    Serial.print(",median:");
    Serial.print(dist_median);
    Serial.print(",Max:");
    Serial.println(_DIST_MAX);

    digitalWrite(PIN_LED,
        (dist_raw < _DIST_MIN || dist_raw > _DIST_MAX) ? HIGH :
LOW);
}

float getMedian(float data[], int size) {
    float sorted[MEDIAN_N];
    for (int i = 0; i < size; i++) sorted[i] = data[i];
    for (int i = 0; i < size - 1; i++) {
        for (int j = i + 1; j < size; j++) {
            if (sorted[i] > sorted[j]) {
                float temp = sorted[i];
                sorted[i] = sorted[j];
                sorted[j] = temp;
            }
        }
    }
    if (size % 2) return sorted[size / 2];
    return (sorted[size / 2 - 1] + sorted[size / 2]) / 2.0;
}

float USS_measure(int TRIG, int ECHO) {
    digitalWrite(TRIG, HIGH);
    delayMicroseconds(PULSE_DURATION);
    digitalWrite(TRIG, LOW);
    return pulseIn(ECHO, HIGH, TIMEOUT) * SCALE;
}
```